Introduction to Queues

What is a Queue?

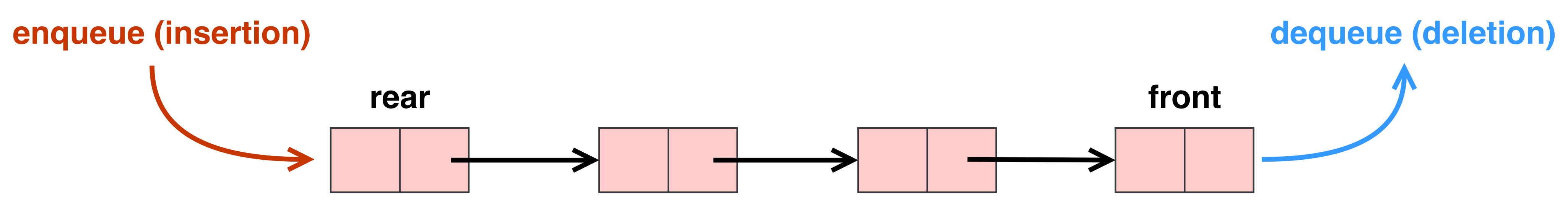
Imagine you're at your favorite coffee shop. It's rush hour, and there's a line of people waiting to place their orders. This line is a perfect real-life example of a Queue! In this line, the person who has been waiting the longest gets served first (we call this the "**First In, First Out**" or **FIFO principle**). Just like this, a Queue in computer science is a type of data structure where the element that enters first is the one that gets accessed first.

Now, why should you care about Queues? Well, Queues are an essential data structure in computer science. They help us solve complex problems and are widely used in various fields, from handling processes in an operating system to managing data packets in networking. By the end of this chapter, you'll see just how valuable understanding Queues can be!

Basic Terminology

To get comfortable with Queues, we need to grasp some key terms.

- **Enqueue**: This is like adding a person to the end of our coffee shop line. In Queues, when we add an element at the end, we call it "Enqueue."
- **Dequeue**: Remember how the first person in line gets their coffee first? Similarly, removing an element from the front of the Queue is called "Dequeue."



A Queue (FIFO)

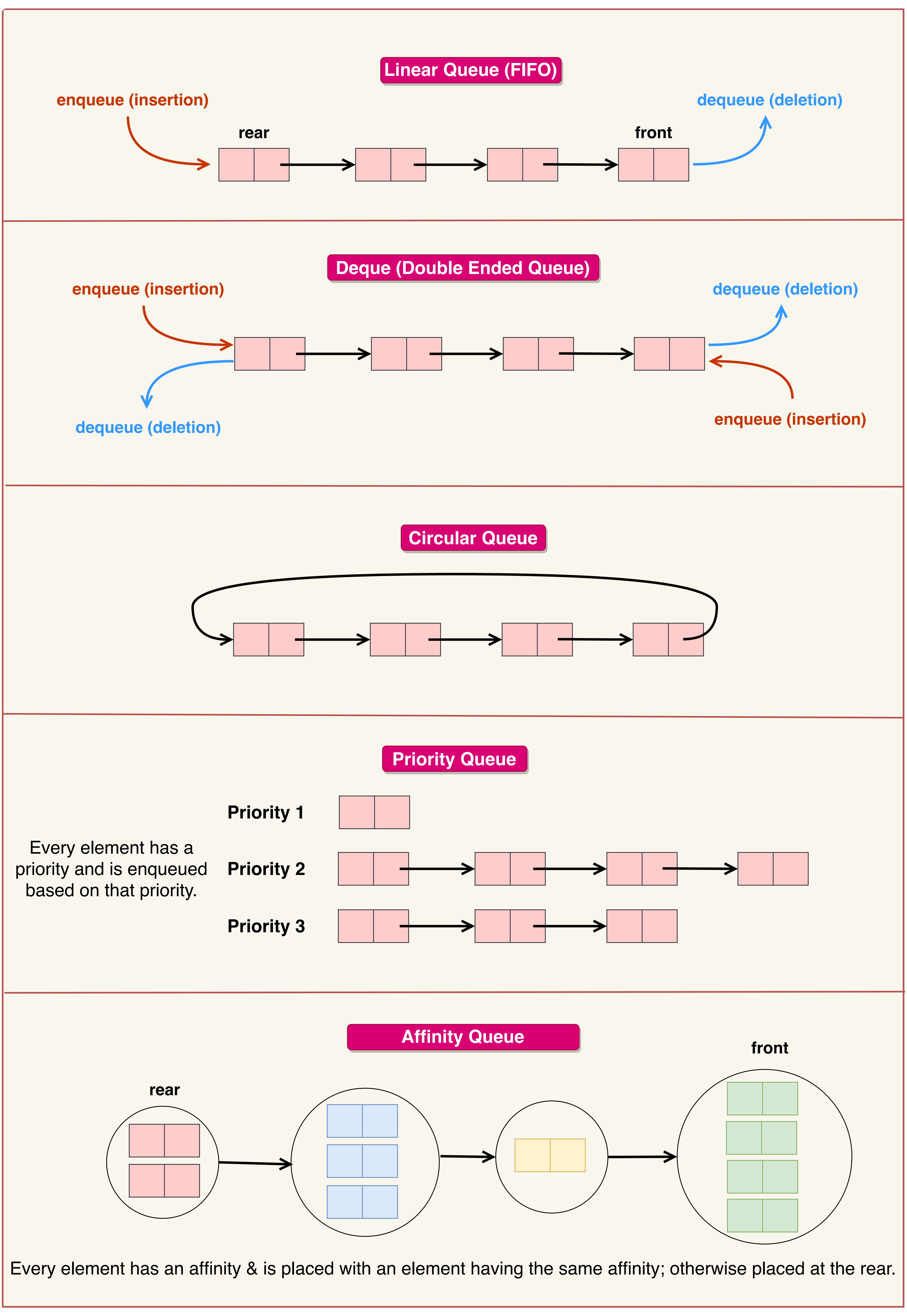
- **Front**: This is the start of the Queue, where the first element resides.
- **Rear**: This is the end of the Queue, where the last element is placed.

When you think about it, it's pretty simple, right? These are the basics, and once you've got a firm grip on them, everything else about Queues will fall into place!

Types of Queues

You might think that all Queues are the same, but that's not quite the case. Just like we have different types of lines (a rush-hour coffee shop line versus a less crowded post office line, for example), we also have different types of Queues. Let's look at a few of these:

1. **Simple Queue**: This is the most basic type. Elements are inserted at the rear and removed from the front, following the FIFO principle.
2. **Deque** (**Double Ended Queue**): Here, elements can be added or removed from both ends of the Queue. Think of it as having a coffee shop line where orders can be taken from both ends!
3. **Circular Queue**: Imagine if our coffee line was in a circle, and once the last person in line gets their coffee, they can directly join the line again at the front. That's a Circular Queue - the last element points to the first, making a loop!
4. **Priority Queue**: In this type, not all elements are equal. Some are considered more important and get to jump the line (like a VIP lane at a concert).
5. **Affinity Queue**: In this type, every element has an affinity & is placed with an element having the same affinity; otherwise placed at the rear.



Queue Types

Each of these types of Queues serves different needs and purposes, and we'll explore them in more detail in the upcoming lessons. Understanding them will expand your problem-solving toolbox and make you a stronger programmer.

← Previous
Solution: Make The String Great
Next →
Working With Simple Queues
☐ Mark as Completed